



DISTRIBUTED COMPUTING

LECTURE # 1

By,
Dr. Ali Akbar Siddique

INTRODUCTION TO PARALLEL AND DISTRIBUTED COMPUTING

- **Definition of Parallel Computing:** Parallel computing is a type of computation in which many calculations or processes are carried out simultaneously, leveraging multiple processing units.
- **Definition of Distributed Computing:** Distributed computing refers to a model where computing tasks are divided across multiple interconnected computers, working together to achieve a common goal.
- **Importance in Modern Computing:**
 - **Performance Enhancement:** Enables faster processing by distributing workloads.
 - **Scalability:** Easily handles increasing amounts of work by adding more computing resources.
 - **Fault Tolerance:** Reduces the risk of system failure by distributing tasks across multiple machines.
 - **Efficiency in Resource Utilization:** Optimizes the use of computational power across various hardware components.

KEY CONCEPTS

- **Concurrency:**
 - Concurrency is the ability of a system to handle multiple tasks in progress at the same time.
 - Tasks may not necessarily be executed simultaneously but make progress together.
 - Example: Running multiple applications on a computer, where the CPU switches between tasks.
- **Parallelism:**
 - Parallelism refers to the simultaneous execution of multiple tasks to achieve faster processing.
 - Requires multiple processing units (e.g., multi-core processors, GPUs).
 - Example: Performing matrix operations using multiple CPU cores simultaneously.
- **Distributed Systems:**
 - A distributed system is a network of independent computers that collaborate to solve a problem.
 - Each machine works on a part of the problem and communicates with others.
 - Example: Cloud computing environments like AWS, Google Cloud, and Microsoft Azure.

CONCURRENCY VS. PARALLELISM

- **Concurrency:**

- Concurrency refers to multiple tasks making progress at overlapping time periods.
- The system rapidly switches between tasks, creating an illusion of simultaneous execution.
- Example: A web server handling multiple client requests concurrently using asynchronous I/O.

- **Parallelism:**

- Parallelism involves executing multiple tasks simultaneously.
- Requires multiple cores or processing units to run computations in parallel.
- Example: Image processing using a GPU, where different sections of an image are processed simultaneously.

CONCURRENCY VS. PARALLELISM

Aspect	Concurrency	Parallelism	Aspect
Execution	Tasks appear to run at the same time but are actually interleaved	Tasks run at the same time on multiple processors	Execution
Dependencies	Often involves shared resources requiring synchronization	Tasks are independent and run without interference	Dependencies
Example	A user switching between multiple browser tabs	A video rendering engine using multi-core processing	Example



DISTRIBUTED SYSTEMS

- A distributed system consists of multiple independent computers working together as a unified system.
- These computers communicate over a network to share resources, perform computations, or provide services.

Characteristics:

- **Decentralization:** No single point of control; workload is distributed among multiple nodes.
- **Scalability:** Can handle increasing workloads by adding more machines.
- **Fault Tolerance:** If one node fails, others can continue functioning, improving system reliability.

Key Components:

- **Nodes:** Individual computing devices participating in the system.
- **Communication Network:** Connects the nodes for data exchange.
- **Middleware:** Software layer enabling coordination between distributed resources.

Examples:

- **Cloud Computing:** Services like AWS, Google Cloud, and Azure use distributed infrastructure.
- **Blockchain Networks:** Bitcoin and Ethereum operate on decentralized distributed systems.
- **Distributed Databases:** Systems like Apache Cassandra and Google Spanner ensure high availability and scalability.



WHY PARALLEL AND DISTRIBUTED COMPUTING?

1. Performance Enhancement

- Traditional sequential processing has limitations in speed and efficiency.
- Parallel and distributed computing enable tasks to be executed simultaneously, reducing computation time.
- **Example:** A weather simulation that would take 10 hours on a single CPU can be completed in minutes using parallel processing across multiple processors.

2. Efficient Resource Utilization

- By distributing workloads across multiple processors or systems, computational resources are used optimally.
- Avoids bottlenecks by balancing loads across different machines.
- **Example:** Cloud computing providers allocate virtual machines dynamically to optimize performance and reduce idle hardware usage.

3. Scalability for Large-Scale Computations

- Parallel and distributed systems can scale horizontally (by adding more machines) or vertically (by increasing processing power).
- Handles massive datasets and complex computations effectively.
- **Example:** Google's search engine processes billions of search queries daily by distributing tasks across thousands of servers worldwide.



WHY PARALLEL AND DISTRIBUTED COMPUTING?

4. Fault Tolerance and Reliability

- Distributed systems ensure redundancy by replicating data and processing tasks across multiple nodes.
- If a node fails, others can take over, minimizing downtime.
- **Example:** Netflix uses a distributed cloud-based infrastructure to ensure seamless video streaming even if some servers fail.

5. Cost-Effectiveness

- Organizations can use distributed computing with commodity hardware instead of expensive supercomputers.
- Cloud platforms offer scalable computing resources on-demand, reducing costs.
- **Example:** A startup can use Amazon Web Services (AWS) to rent computing power instead of investing in costly data centers.



REAL-WORLD APPLICATIONS OF PARALLEL AND DISTRIBUTED COMPUTING

1. Big Data Processing

- The increasing volume of data requires powerful computational methods to process and analyze massive datasets efficiently.
- Parallel and distributed computing enable real-time data processing and analytics.
- **Technologies Used:**
 - **Apache Hadoop:** Uses the MapReduce framework to distribute data processing across clusters of computers.
 - **Apache Spark:** Performs in-memory distributed computing for faster data analysis.
- **Example:**
 - **Netflix** uses Hadoop and Spark to analyze user preferences and provide personalized movie recommendations.



REAL-WORLD APPLICATIONS OF PARALLEL AND DISTRIBUTED COMPUTING

2. Artificial Intelligence (AI) and Machine Learning

- Training deep learning models requires processing large datasets and performing complex mathematical computations.
- GPUs and TPUs (Tensor Processing Units) enable parallel computation, reducing training time.
- **Technologies Used:**
 - **GPUs (CUDA, TensorFlow, PyTorch):** Perform matrix multiplications in parallel.
 - **Distributed Deep Learning:** Frameworks like Horovod and TensorFlow Distributed speed up training across multiple machines.
- **Example:**
 - **OpenAI's ChatGPT** and Google's **DeepMind AlphaGo** use distributed computing to train AI models on massive datasets.



REAL-WORLD APPLICATIONS OF PARALLEL AND DISTRIBUTED COMPUTING

3. Cloud Computing

- Cloud platforms use distributed computing to provide on-demand computing resources.
- Users can scale applications dynamically without investing in expensive hardware.
- **Cloud Service Providers:**
 - **Amazon Web Services (AWS):** Offers computing power (EC2), storage (S3), and databases.
 - **Google Cloud Platform (GCP):** Provides AI and machine learning services.
 - **Microsoft Azure:** Supports virtual machines and enterprise applications.
- **Example:**
 - **Instagram, Dropbox, and Spotify** use cloud computing to store and process massive amounts of user data across multiple servers.



An abstract graphic on the left side of the slide featuring a dark blue background with a complex network of white, glowing circuit lines. These lines are interconnected with small, bright yellow and orange circular nodes, creating a sense of digital connectivity and data flow.

PARALLEL AND DISTRIBUTED COMPUTING IN AI

- Artificial Intelligence (AI) requires enormous computational power to train deep learning models and process vast amounts of data. Parallel and distributed computing enable efficient AI training, optimization, and real-time inference.

1. Why AI Needs Parallel and Distributed Computing?

- **Massive Data Processing:** AI models require large datasets (e.g., ImageNet, GPT datasets).
 - **Computational Intensity:** Deep learning involves billions of mathematical operations.
 - **Real-Time Inference:** AI applications like autonomous vehicles and facial recognition require fast decision-making.
- 
- A small, solid red circle with a thin white outline, located in the bottom right corner of the slide.

An abstract graphic on the left side of the slide featuring a dark blue background with a complex network of white, glowing circuit lines. These lines are interconnected with various geometric shapes, including circles, squares, and diamonds, some of which are highlighted with a yellow glow. The overall effect is reminiscent of a high-tech circuit board or a neural network diagram.

PARALLEL AND DISTRIBUTED COMPUTING IN AI

2. Training Deep Learning Models with Parallel Computing

- Training AI models involves performing millions of matrix multiplications.
- GPUs (Graphics Processing Units) and TPUs (Tensor Processing Units) execute these operations in parallel.
- **Key Technologies:**
 - **CUDA (Compute Unified Device Architecture):** Enables GPU acceleration for AI tasks.
 - **TensorFlow and PyTorch:** Use multi-core processing for deep learning.
 - **NVIDIA Tensor Cores:** Optimize AI computations for faster training.
- **Example:**
 - **Google's AlphaGo** used parallel computing with TPUs to train its reinforcement learning model.

An abstract graphic on the left side of the slide featuring a dark blue background with glowing white and yellow circuit-like lines and nodes, creating a sense of digital connectivity and data flow.

PARALLEL AND DISTRIBUTED COMPUTING IN AI

3. AI in Edge and IoT Devices

- AI models can be optimized for edge devices (smartphones, IoT sensors).
- Edge AI reduces reliance on cloud computing by processing data locally.
- **Example:**
- **Tesla's Autopilot** uses parallel processing in AI chips for real-time driving decisions.



Tool/Framework	Purpose	Example Use Case
CUDA (Compute Unified Device Architecture)	Enables parallel computing using NVIDIA GPUs.	Deep learning model training with TensorFlow.
MPI (Message Passing Interface)	Manages communication in distributed systems.	Weather forecasting simulations using supercomputers.
OpenMP (Open Multi-Processing)	Supports shared-memory parallelism in C, C++, and Fortran.	Multi-threaded scientific computing applications.
Apache Hadoop	Distributed storage and processing for big data.	Processing large datasets in cloud environments.
Apache Spark	In-memory distributed computing for big data analytics.	Real-time data processing for financial transactions.
Ray	Parallel computing framework for AI and machine learning workloads.	Large-scale reinforcement learning training.

TOOLS FOR PARALLEL AND DISTRIBUTED COMPUTING

- Parallel and distributed computing rely on specialized tools and frameworks to manage computations efficiently. These tools enable developers to optimize performance, handle large-scale data, and coordinate distributed tasks.
- Several tools and frameworks are commonly used in parallel and distributed computing. Each tool serves a different purpose, from GPU acceleration to large-scale distributed processing.



KEY DIFFERENCES BETWEEN THE TOOLS

Aspect	CUDA	MPI	OpenMP	Hadoop	Spark	Ray
Computing Model	Parallel (GPU)	Distributed	Parallel (CPU)	Distributed	Distributed	Parallel & Distributed
Best For	AI, Deep Learning	Supercomputing	Scientific Computing	Big Data Processing	Real-Time Big Data	AI & ML Workloads
Memory Access	Shared GPU Memory	Distributed Memory	Shared Memory	Distributed Storage	In-Memory Processing	Distributed Execution
Ease of Use	Moderate	Complex	Easy	Moderate	Easy	Easy
Scalability	High	High	Medium	Very High	Very High	High



WHEN TO USE EACH TOOL

- **CUDA** → Best for AI, deep learning, and GPU-accelerated computations.
- **MPI** → Best for large-scale **distributed simulations** in scientific research.
- **OpenMP** → Best for multi-threaded **CPU-based parallel processing**.
- **Hadoop** → Ideal for **big data storage and batch processing**.
- **Spark** → Suitable for **real-time analytics and big data workflows**.
- **Ray** → Ideal for **AI model training, reinforcement learning, and distributed Python workloads**.



Thank
you

